

OBJECT-ORIENTED SOFTWARE ENGINEERING (OOSE)

Regulation 2025 (R2025) – CS25C10

Exam-oriented notes covering the six major units of the Anna University R2025 syllabus. The syllabus includes software process and Agile development, requirements/UML, software design, design patterns and architecture, testing and maintenance, and project management/DevOps. ■cite■turn1search2■turn1search3■

UNIT I – SOFTWARE PROCESS AND AGILE DEVELOPMENT

1. Software Engineering: Software engineering is the systematic application of engineering principles, methods and tools to develop, operate, test and maintain software.

Objectives: Produce quality software; satisfy user requirements; reduce cost/time; improve reliability, maintainability and efficiency; manage complex projects.

Software Process: A structured set of activities used to develop and maintain software. Typical activities are communication/requirements, planning, modeling/design, construction and deployment/maintenance.

Process Models: **Waterfall:** sequential phases; suitable when requirements are stable. **Incremental:** delivers the system in increments. **Prototyping:** builds an early model to clarify requirements. **Spiral:** iterative and risk-driven. **Evolutionary:** develops the system through repeated refinement.

Agile Process: Agile is iterative and incremental. It emphasizes working software, customer collaboration, frequent delivery, responding to change and continuous improvement.

Agile Manifesto – 4 values: Individuals and interactions over processes and tools; working software over comprehensive documentation; customer collaboration over contract negotiation; responding to change over following a plan.

Extreme Programming (XP): An Agile method emphasizing frequent releases, customer feedback, simple design, refactoring, pair programming, test-driven development, continuous integration and collective code ownership.

Exam focus: Compare Waterfall and Agile; explain process models; explain Agile values/principles; explain XP practices and advantages.

UNIT II – REQUIREMENTS ANALYSIS AND SPECIFICATION

Requirements: Requirements describe what the system should provide and its constraints. Functional requirements specify services/behaviour; non-functional requirements specify qualities such as performance, security, usability and reliability.

SRS: Software Requirements Specification is a structured document describing functional/non-functional requirements, interfaces, constraints and acceptance-related information. A good SRS should be clear, consistent, complete, verifiable and traceable.

Formal Specification: Uses mathematical notation to describe system behaviour precisely. It helps remove ambiguity and can support verification.

Finite State Machine (FSM): A model with states and transitions. A transition occurs when an input/event is received and may produce an output or action. Useful for systems with state-dependent behaviour.

Petri Nets: A graphical/mathematical model using places, transitions, tokens and arcs. Useful for modeling concurrency, synchronization and resource sharing.

UML: Unified Modeling Language is a standard visual modeling language for specifying, visualizing and documenting software systems.

Use Case Diagram: Shows actors and their interactions with system functions (use cases). Main relationships include association, include and extend.

Class Diagram: Shows classes, attributes, operations and relationships such as association, aggregation, composition, generalization and dependency.

Interaction Diagrams: Sequence diagrams emphasize time/order of messages; communication diagrams emphasize object links and message exchange.

Activity Diagram: Models workflow or business/process activities, including decisions, parallel activities and start/end points.

State Chart Diagram: Shows states of an object and transitions caused by events.

DFD: Data Flow Diagram represents processes, data flows, data stores and external entities. It focuses on functional/data movement rather than object structure.

CASE Tools: Computer-Aided Software Engineering tools support activities such as requirements management, UML modeling, documentation, testing and project management.

UNIT III – SOFTWARE DESIGN

Software Design: Transforms requirements into a blueprint for implementation. Good design aims for correctness, simplicity, maintainability, flexibility, reusability and efficiency.

Design Metrics: Measures used to evaluate design characteristics such as complexity, coupling, cohesion, size and maintainability.

Modularity: Dividing a system into manageable modules with well-defined responsibilities and interfaces.

Coupling: Degree of dependency between modules. Lower coupling is generally preferred.

Types of Coupling: Common forms range from stronger/worse coupling such as content/common coupling to weaker/better forms such as data/message coupling. The goal is to minimize unnecessary dependencies.

Cohesion: Degree to which elements within a module belong together. High cohesion is preferred.

Types of Cohesion: Common forms include coincidental, logical, temporal, procedural, communicational, sequential and functional cohesion. Functional cohesion is generally the strongest.

Top-Down Design: Starts with the overall system and decomposes it into smaller subsystems/modules.

Bottom-Up Design: Starts with reusable low-level components and combines them to build higher-level functionality.

Hybrid Design: Combines top-down decomposition with bottom-up component reuse.

Functional-Oriented Design: Organizes the system around functions/processes and data transformations.

Object-Oriented Design: Organizes the system around objects/classes that encapsulate state and behaviour. It uses abstraction, encapsulation, inheritance and polymorphism.

Exam focus: Explain coupling/cohesion with types; compare top-down and bottom-up design; explain modularity; compare functional-oriented and object-oriented design.

UNIT IV – DESIGN PATTERNS AND ARCHITECTURAL STYLES

Design Pattern: A reusable, proven solution template for a recurring software design problem. It is not a complete program.

MVC: Model stores data/business logic; View presents information; Controller handles input and coordinates Model and View.

Observer Pattern: Defines a one-to-many dependency so observers are notified when the subject changes.

Strategy Pattern: Encapsulates interchangeable algorithms behind a common interface so the algorithm can be selected independently.

Adapter Pattern: Converts the interface of an existing class into the interface expected by a client.

Command Pattern: Encapsulates a request as an object, enabling parameterization, queuing, logging and undo operations.

Facade Pattern: Provides a simple unified interface to a complex subsystem.

Proxy Pattern: Provides a substitute or representative for another object to control access, add security, caching, lazy loading or remote access.

Publish-Subscribe: Publishers send messages/events to a broker or topic; subscribers receive messages without direct coupling to publishers.

Architectural Styles: **Layered:** system divided into layers. **Client-Server:** clients request services from servers. **Tiered:** separates presentation, application and data tiers. **Pipe-and-Filter:** processing is organized as filters connected by data pipes.

User Interface Design: Focuses on usability, consistency, feedback, simplicity, accessibility, error prevention and user control.

Case Study Approach: Identify requirements, select architecture/patterns, create UML models, design components, test the solution and justify design choices.

UNIT V – SOFTWARE TESTING AND MAINTENANCE

Software Testing: Testing is the systematic evaluation of software to find defects and provide confidence that requirements are satisfied.

Unit Testing: Tests individual functions/classes/modules, usually in isolation.

Black-Box Testing: Tests external behaviour without knowing internal implementation. Techniques include equivalence partitioning and boundary value analysis.

White-Box Testing: Uses knowledge of internal code/control flow. Techniques include statement, branch and path-oriented testing.

Integration Testing: Tests interactions among combined modules/components.

System Testing: Tests the complete integrated system against specified requirements.

Regression Testing: Re-runs relevant tests after changes to ensure existing functionality has not been broken.

Debugging: Process of locating, analyzing and correcting the cause of a failure/defect.

Program Analysis: Examines program structure and behaviour to detect defects, security issues or other properties.

Symbolic Execution: Executes a program using symbolic values instead of only concrete inputs and derives path conditions to explore behaviours.

Model Checking: Automatically verifies whether a formal model satisfies specified properties, often by exploring system states.

Software Maintenance: Modification after delivery to correct faults, adapt to environmental changes, improve performance/usability, or prevent future problems. Types commonly include corrective, adaptive, perfective and preventive maintenance.

Exam focus: Compare black-box and white-box testing; explain unit/integration/system/regression testing; explain debugging, symbolic execution and model checking; explain maintenance types.

UNIT VI – PROJECT MANAGEMENT AND DEVOPS

Software Project Management: Planning, organizing, monitoring and controlling people, resources, schedule, cost, scope, quality and risks to achieve project objectives.

Software Configuration Management (SCM): Controls and tracks changes to software artifacts. Key ideas include configuration items, version control, baselines, change control and status accounting.

Project Scheduling: Allocates tasks and resources over time. Important tools include milestone charts, Gantt charts, PERT and CPM.

Gantt Chart: Bar chart showing tasks against time. Useful for schedules, progress tracking and dependencies.

PERT: Program Evaluation and Review Technique uses three time estimates—optimistic, most likely and pessimistic—to estimate activity duration under uncertainty.

CPM: Critical Path Method identifies the longest-duration path through a project network. Activities on the critical path have little or no scheduling slack.

DevOps: Combines development and operations practices to deliver software rapidly and reliably through automation, collaboration, continuous integration and continuous delivery/deployment.

Cloud as a Platform: Cloud infrastructure/platform services provide scalable resources for development, testing, deployment and operations.

Deployment Pipeline: Automated flow that builds, tests, packages and deploys software through stages.

CI/CD: Continuous Integration frequently integrates code and runs automated checks. Continuous Delivery/Deployment automates release preparation or deployment.

Overall Architecture: DevOps architecture commonly connects source control, build, automated tests, artifact repositories, deployment environments, monitoring and feedback.

Exam focus: Explain SCM; Gantt/PERT/CPM; critical path; DevOps lifecycle; CI/CD and deployment pipeline; cloud as a platform.

LAST-MINUTE REVISION CHECKLIST

1. Learn all six unit headings and the definition of each major term.
2. Practice UML: Use Case, Class, Sequence/Communication, Activity and State Chart diagrams.
3. Prepare comparisons: Waterfall vs Agile, black-box vs white-box, top-down vs bottom-up, coupling vs cohesion, PERT vs CPM.
4. Practice design patterns: MVC, Observer, Strategy, Adapter, Command, Proxy and Facade.
5. Practice Gantt chart, PERT/CPM and critical-path problems.
6. Remember testing levels: Unit → Integration → System, plus Regression.
7. For long answers, write: definition → diagram/steps → explanation → example → advantages/applications → conclusion.